

INT202

Complexity of Algorithms

Introduction

XJTLU/SAT/INT
SEM2 AY2025-2026

INT202: Complexity of Algorithms

- Module Leader:

Dr. Rui Yang

- Email:

R.Yang@xjtlu.edu.cn

- Lectures:

- SA169 Tue (14:00-15:50)

- Tutorials:

- SA169 Fri (15:00-16:50)

- Assessments:

- 1 In-Class Test in Week 6 (10%)
- 1 In-Class Test in Week 13 (10%)
- 1 Final Exam (80%)

Algorithms and Data Structures

- ❖ A *data structure* is a systematic way of organizing and accessing data.
- ❖ An *algorithm* is a sequence of steps for performing a task in a finite amount of time.

- Navigate to a place
- Transfer Audio/Video format
- Control solar panels
- Generate an image from 2D/3D model
- Root-finding algorithm
- Compression algorithm
- Optimization algorithm
- Rendering algorithm

Algorithms and Data Structures

- What makes a good algorithm?
- How do you measure efficiency?

Our fundamental interest is to design “good” *algorithms* which utilize efficient *data structures*. In this context “good” means *fast* in a way to be clarified during this course.

Algorithm Analysis

- ❖ The analysis of algorithm is the theoretical study of computer program performance and resource usage.
 - ❖ **Primary interest:** Running time (*time-complexity*) of the algorithm and the operations on data structures.
 - ❖ **Secondary interest:** Space (or “memory”) usage (*space-complexity*).
-
- This course focuses on the *mathematical* design and analysis of algorithms and their associated data structures, utilizing relevant mathematical tools to do so.

Experimental Analysis of Algorithms

- ❖ We are interested in *dependency* of the running time or memory requirement on *size* of the input.
- The *size* could mean the number of vertices and edges if we are operating on a graph, the length of a message we're encoding/decoding, and/or the actual length of numbers we're processing in terms of the bits needed to store them in memory.
- To analyze algorithms we sometimes perform *experiments* to empirically observe for example the running time.
- Doing so requires a good choice of sample inputs and an appropriate number of tests (so that we can have *statistical certainty* about our analysis).

Experimental Analysis (cont.)

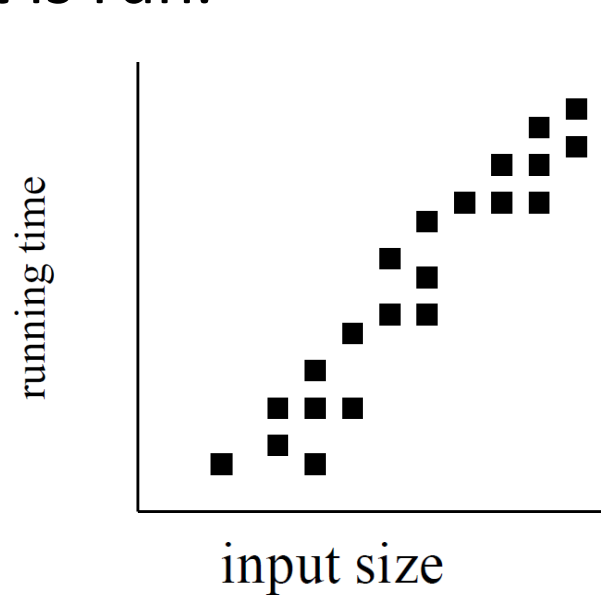
Running-time depends on both the *size* and *instance* of *input* and the algorithm used, as well as the software and hardware environment on which it is run.



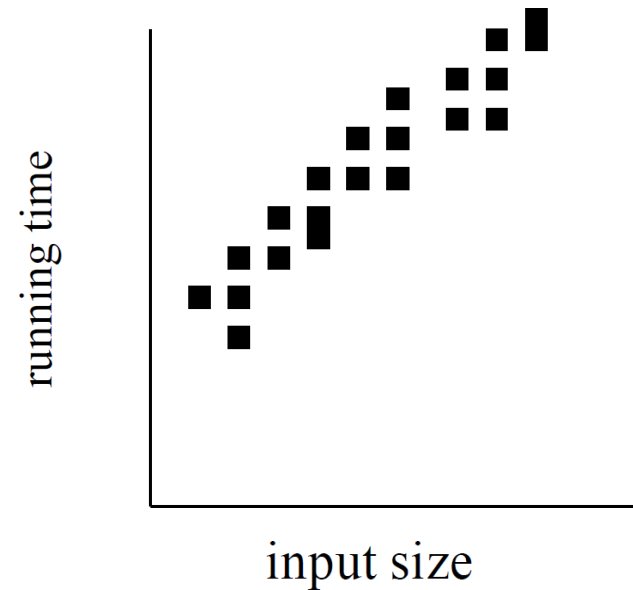
In general, the running time of an algorithm or data structure method increases with the input size

Experimental Analysis (cont.)

Running-time depends on both the *size* and *instance* of input and the algorithm used, as well as the **software and hardware environment** on which it is run.



a fast computer



a slow computer

Limitations of Experimental Analysis

1. Experiments are performed on a limited set of test inputs.
2. Requires all tests to be performed using same hardware and software.
3. Requires implementation and execution of algorithm.

Theoretical Analysis

Benefits over experimental analysis:

1. Can take all possible inputs into account.
2. Can compare efficiency of two (or more) algorithms, independent of hardware/software environment.
3. Involves studying high-level descriptions of algorithms (*pseudo-code*).

Theoretical Analysis (cont.)

- ❖ When we analyze an algorithm in this fashion, we aim to associate a function $f(n)$ to each algorithm, where $f(n)$ characterizes the running-time in terms of some measure of the *input-size*, n .

Typical functions include: n , $\log n$, n^2 , $n \log n$, 2^n ,...

‘Algorithm A runs in time proportional to n ’

Theoretical Analysis (requirements)

For formal theoretical analysis, we need:

1. *A language for* describing algorithms.
2. *Computational model* in which algorithms are executed.
3. *Metric* for measuring performance.
4. A way of characterizing performance.

Pseudo-code

- ❖ Pseudo-code is a high-level description language for algorithms.

Pseudo-code provides more structured description of algorithms.

Allows high-level analysis of algorithms to determine their running time (and memory requirements).

Task: Give a Pseudo-code for the function *Minimum-Element*, which finds the minimum element of a set of numbers.

Pseudo-code

Algorithm Minimum-Element(A)

input: An array A sorting $n \geq 1$ integers

output: minimum element min

$min \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n-1$ **do**

if $min > A[i]$ *then*

$min \leftarrow A[i]$

return min

- Instead of the sign arrow $\leftarrow$ an equal sign can also be used
- Instead of if-then-end if, the curly brackets $\{ \}$ can also be used

Pseudo-code

- ❖ Pseudo-code is a mixture of natural language and high-level programming language (e.g., Java, C, etc.).

Describes a generic implementation of data structures or algorithms.

- ❖ Pseudo-code includes: expressions, declarations, variable initialization and assignments, conditionals, loops, arrays, method calls, etc.

In here, we won't formally define a strict method for giving pseudo-code, but when using it we aim to describe an algorithm in a manner that would allow a competent programmer to translate the pseudo-code into program code without misinterpretation of the algorithm.

Pseudo-code

We define a set of high-level primitive operations that are largely independent from the programming language used.

Primitive operations include the following:

- Assigning a value to a variable
 - Calling a method
 - Performing an arithmetic operation (for example, adding two numbers)
 - Comparing two numbers
 - Indexing into an array
 - Following an object reference
 - Returning from a method.
- ❖ To analyze the running time of an algorithm we *count* the number of operations executed during the course of the algorithm.

Counting Primitive Operations

Algorithm `arrayMax(A,n)`:

input: array *A*

output: maximum element *currentMax*

```
1  currentMax  $\leftarrow$  A[0]
2  for j  $\leftarrow$  1 to n-1 do
3      if currentMax < A[j]
4          then currentMax  $\leftarrow$  A[j]
5  return currentMax
```



How many primitive operations?

Random Access Machine

This approach of simple counting primitive operations gives rise to a computational model called the Random Access Machine.

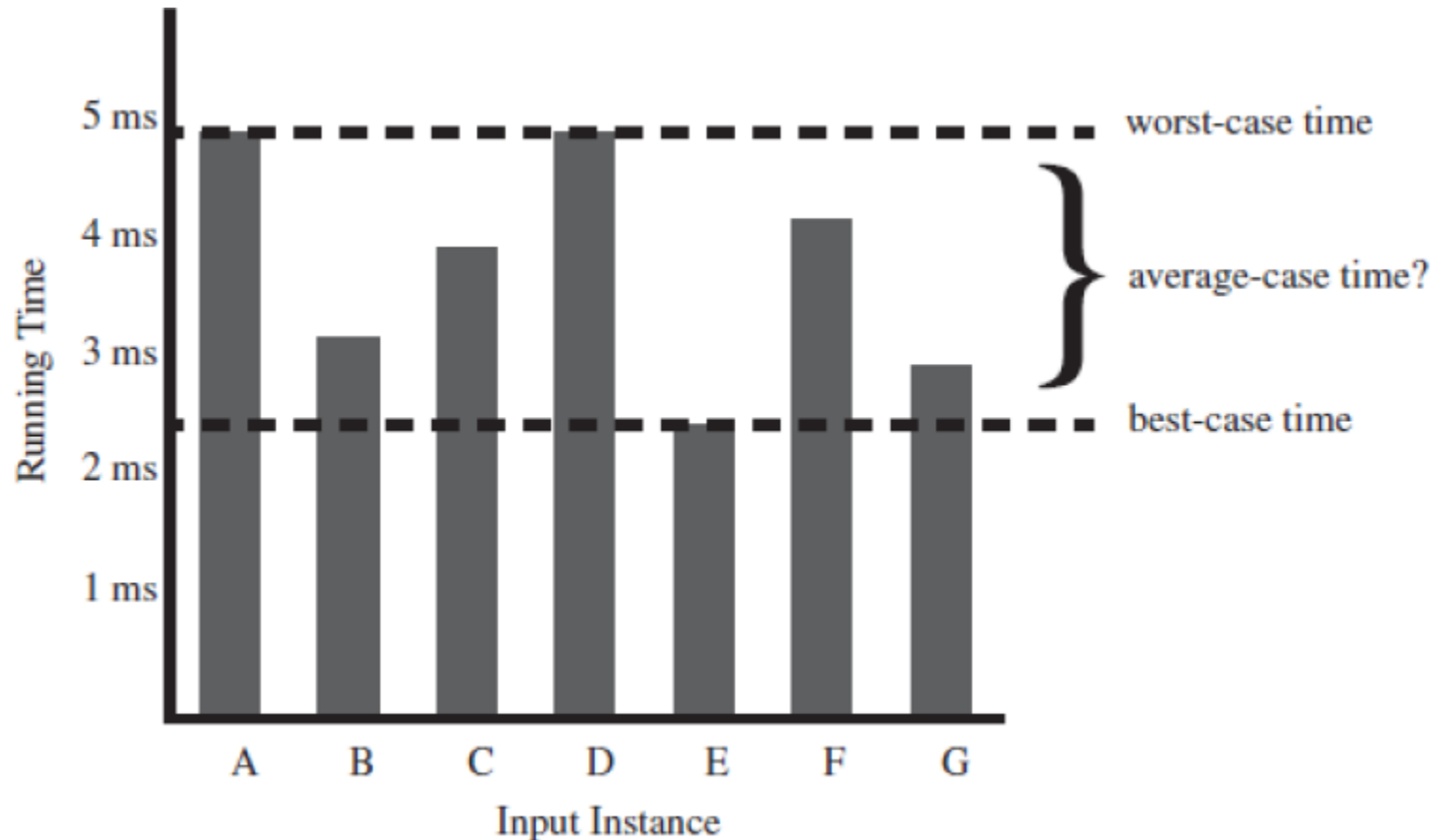
- ❖ CPU connected to a bank of *memory cells*.
- ❖ Each memory cell can store a number, character, or address.

Assumption: primitive operations (like a single addition, multiplication, or comparison) require *constant time* to perform.

(Not necessarily true, e.g. multiplication takes about four times as long to perform on a computer than addition.)

Average- vs. Worst-Case Complexity

An algorithm may run faster on some inputs compared to others.



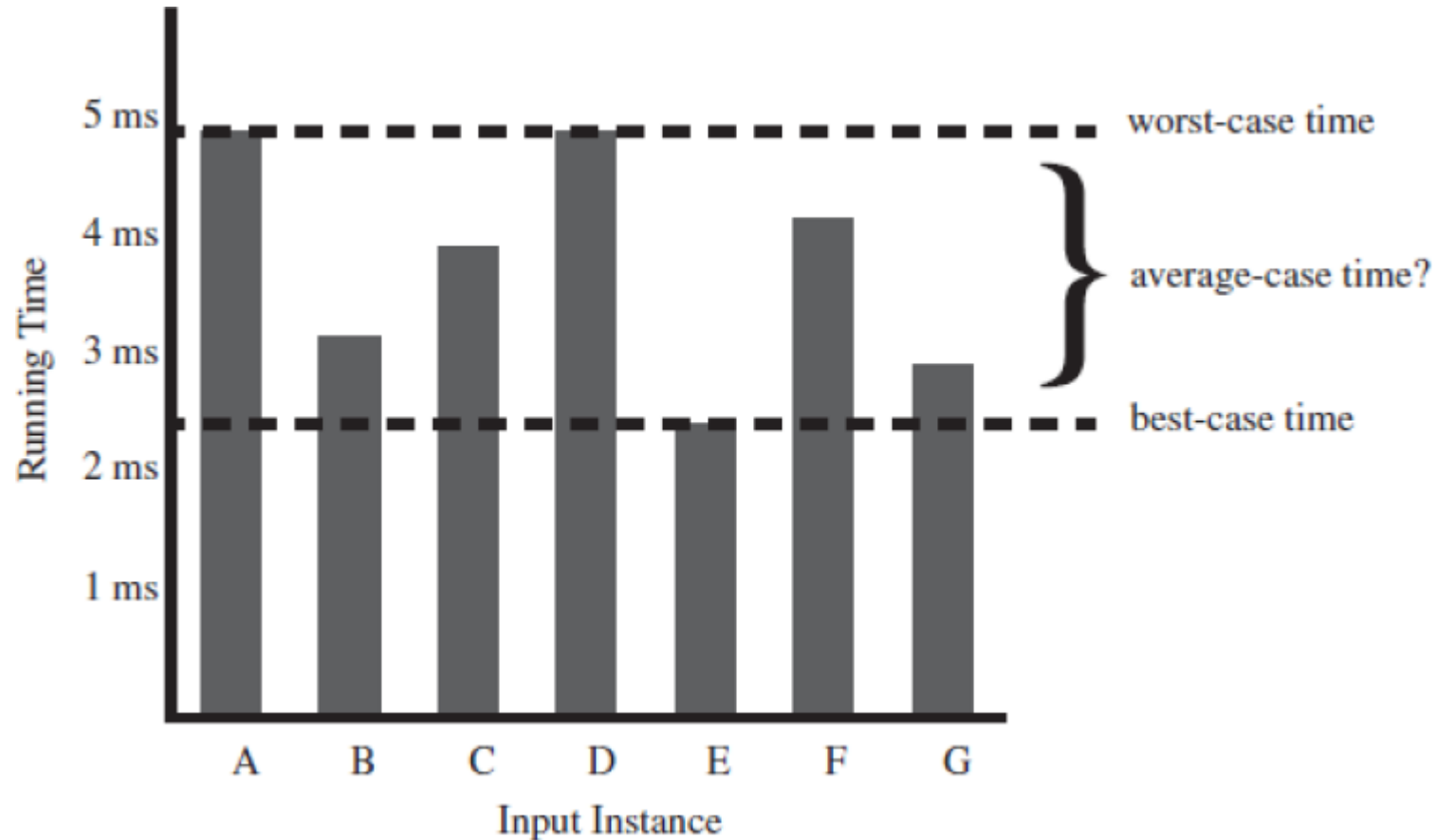
Average- vs. Worst-Case Complexity

An algorithm may run faster on some inputs compared to others.

- ❖ *Average-case* complexity refers to running time as an *average taken over all inputs* of the same size.
- ❖ *Worst-case* complexity refers to running time as the *maximum taken over all inputs* of the same size.

Usually, we're most interested in worst-case complexity.

Average- vs. Worst-Case Complexity (cont.)



An average-case analysis also typically requires that we calculate expected running times based on a given input distribution.

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element

currentMax

```
1  currentMax  $\leftarrow A[0]$ 
2  for  $j \leftarrow 1$  to  $n-1$  do
3      if currentMax  $< A[j]$ 
4          then currentMax  $\leftarrow A[j]$ 
5  return currentMax
```

How many primitive operations?

1. Line 1:

2. Line 2:

3. Line 5:

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element

currentMax

```
1  currentMax ← A[0]
2  for  $j \leftarrow 1$  to  $n-1$  do
3      if currentMax < A[j]
4          then currentMax ← A[j]
5  return currentMax
```

How many primitive operations?

- | | |
|--|-----|
| 1. Line 1: Array indexing + Assignment | 2 |
| 2. Line 2: Initializing j | 1 |
| Verifying $j < n$ | n |
| 3. Line 5: Returning | 1 |

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element

currentMax

1 *currentMax* $\leftarrow$ A[0]

2 **for** $j \leftarrow 1$ **to** $n-1$ **do**

3 **if** *currentMax* < A[j]

4 **then** *currentMax* $\leftarrow$ A[j]

5 **return** *currentMax*

How many primitive operations?

4. Line 3:

5. Line 4:

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element

currentMax

1 *currentMax* $\leftarrow$ A[0]

2 **for** $j \leftarrow 1$ **to** $n-1$ **do**

3 **if** *currentMax* < A[j]

4 **then** *currentMax* $\leftarrow$ A[j]

5 **return** *currentMax*

How many primitive operations?

4. Line 3: Array indexing +Comparing 2(n-1)

5. Line 4: Array indexing +Assignment 2(n-1)

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element

currentMax

1 *currentMax* $\leftarrow$ A[0]

2 **for** $j \leftarrow 1$ **to** $n-1$ **do**

3 **if** *currentMax* < A[j]

4 **then** *currentMax* $\leftarrow$ A[j]

5 **return** *currentMax*

Worst case

How many primitive operations?

4. Line 3

5. Line 4

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element

currentMax

```
1  currentMax  $\leftarrow$  A[0]
2  for  $j \leftarrow 1$  to  $n-1$  do
3      if currentMax < A[j]
4          then currentMax  $\leftarrow$  A[j]
5  return currentMax
```

Best case ?

How many primitive operations?

4. Line 3

5. Line 4

Counting Primitive Operations

Algorithm arrayMax(A,n):

input: array A

output: maximum element *currentMax*

		Count
1	<i>currentMax</i> $\leftarrow$ A[0]	Array indexing + Assignment 2
2	for <i>j</i> $\leftarrow$ 1 to <i>n-1</i> do	Initializing <i>j</i> Verifying <i>j</i> < <i>n</i> 1+ <i>n</i>
3	if <i>currentMax</i> <A[<i>j</i>]	Array indexing + Comparing 2(<i>n</i> -1)
4	then <i>currentMax</i> $\leftarrow$ A[<i>j</i>]	Array indexing + Assignment 2(<i>n</i> -1)
		Incrementing the counter 2(<i>n</i> -1)
5	return <i>currentMax</i>	Returning 1

How many primitive operations?

Best case: $2+1+n+4(n-1)+1=5n$

Worst case: $2+1+n+6(n-1)+1=7n-2$

Recursive Algorithms

- Recursion involves a procedure calling itself to solve *sub-problems* of a smaller size. These smaller subproblems can then be combined in some way to get a solution to a larger problem.
- Recursive procedures require a *base case* that can be solved directly without using recursion.

Recurrence Relations

❖ *Recurrence relations* sometimes allow us to define the running-time of an algorithm in the form of an equation.

Suppose that $T(n)$ denotes the running time of algorithm on input of size n . Then we might be able to characterize $T(n)$ in terms of, say, $T(n - 1)$. For example, we might be able to show that

$$T(n) = \begin{cases} 3 & \text{if } n = 1 \\ T(n - 1) + 7 & \text{for } n \geq 2 \end{cases}$$

Ideally, given such a relationship we would then want to express this recurrence relation in a *closed form*.

In the example, we can show that

$$T(n) = 7(n - 1) + 3 = 7n - 4.$$

3 10 17 24 ...

Recurrence Relations (cont.)

- Recurrence relations may appear in many forms. Some examples include:

1. $C(n) = 3 \cdot C(n - 1) + 2 \cdot C(n - 2) + C(n - 3)$ where

$$C(1) = 1, C(2) = 3, C(3) = 5$$

2. The Fibonacci numbers

Recursion example: Fibonacci Numbers

The Fibonacci numbers are defined as the sequence

$f_1 = f_2 = 1$, and $f_n = f_{n-1} + f_{n-2}$, for $n \geq 3$. They can be found using the following pseudo-code that computes them recursively.

Problem: Write a piece of pseudocode to compute Fibonacci numbers, $n=50$.

The terms of the Fibonacci sequence are: 1,1,2,3,5,8,13,21,34.

Problem

Algorithm: fibonacci numbers

Input: upper limit n

Output: The n-th term of Fibonacci

```
int fib (int n){
```

```
  if n <=2
```

```
    return 1;
```

```
  else
```

```
    return fib(n-1)+fib(n-2);
```

```
}
```

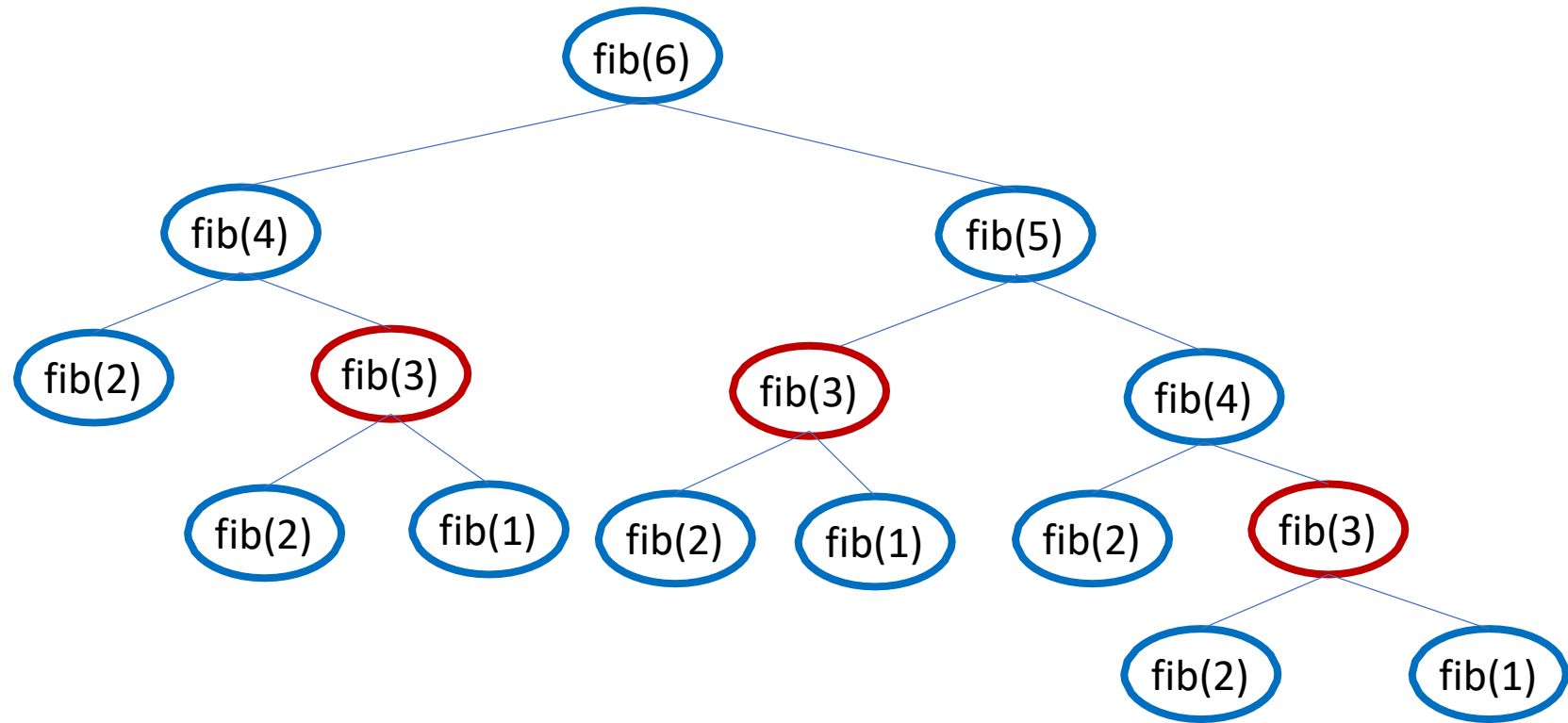
Recursive Algorithms: A Word of Caution...

While recursive algorithms are often “simpler” to write than a non-recursive version, there are often reasons to avoid them.

- ❖ In many situations, the smaller subproblems might be solved *repeatedly* during execution of the recursive algorithm.

Recursive Algorithms: A Word of Caution...

To compute $\text{Fibonacci}(n)$, we must compute $\text{Fibonacci}(n - 1)$ and $\text{Fibonacci}(n - 2)$. **Both** of these function calls must then compute $\text{Fibonacci}(n - 3)$ and $\text{Fibonacci}(n - 4)$, etc. This repetition of work can massively increase the overall running time of the algorithm.



Recursive Algorithms: A Word of Caution...

To compute $\text{Fibonacci}(n)$, we must compute $\text{Fibonacci}(n - 1)$ and $\text{Fibonacci}(n - 2)$. **Both** of these function calls must then compute $\text{Fibonacci}(n - 3)$ and $\text{Fibonacci}(n - 4)$, etc. This repetition of work can massively increase the overall running time of the algorithm.

- ❖ Because of the above phenomena, a recursive algorithm could also be impossible to perform on a computer (for large input values) because the repeated function calls might exhaust the memory of the machine.

Recursive Algorithms

- Recursion involves a procedure calling itself to solve *sub-problems* of a smaller size. These smaller subproblems can then be combined in some way to get a solution to a larger problem.
- Recursive procedures require a *base case* that can be solved directly without using recursion.

Exercise

- ❖ Write pseudo-code for a non-recursive method to compute $Fibonacci(n)$, assuming that n is a positive integer.

Hint: Avoid the repeated computations mentioned above, by starting from the beginning of the sequence and “working up” to get the term you want.

Exercise

Algorithm: fibonacci numbers

Input: upper limit Nmax

Int f(int Nmax)

{

f1 $\leftarrow$ 1;

f2 $\leftarrow$ 1;

for n $\leftarrow$ 3:(Nmax-2){

fn $\leftarrow$ f2 + f1;

f1 $\leftarrow$ f2;

f2 $\leftarrow$ fn;

return fn;

}}

Asymptotic notation

❖ *Asymptotic notation* allows characterization of the *main factors* affecting running time.

Used in a *simplified analysis* that estimates the number of primitive operations executed *up to a constant factor*.

Such notation lets us compare the running times of two algorithms.

Importance of asymptotics

Maximum size allowed for an input instance for various running times to be solved in 1 second, 1 minute and 1 hour, assuming a 1MHz machine:

Running Time	Maximum problem size (n)		
	1 second	1 minute	1 hour
$400n$	2,500	150,000	9,000,000
$20n \log n$	4,096	166,666	7,826,087
$2n^2$	707	5,477	42,426
n^4	31	88	244
2^n	19	25	31

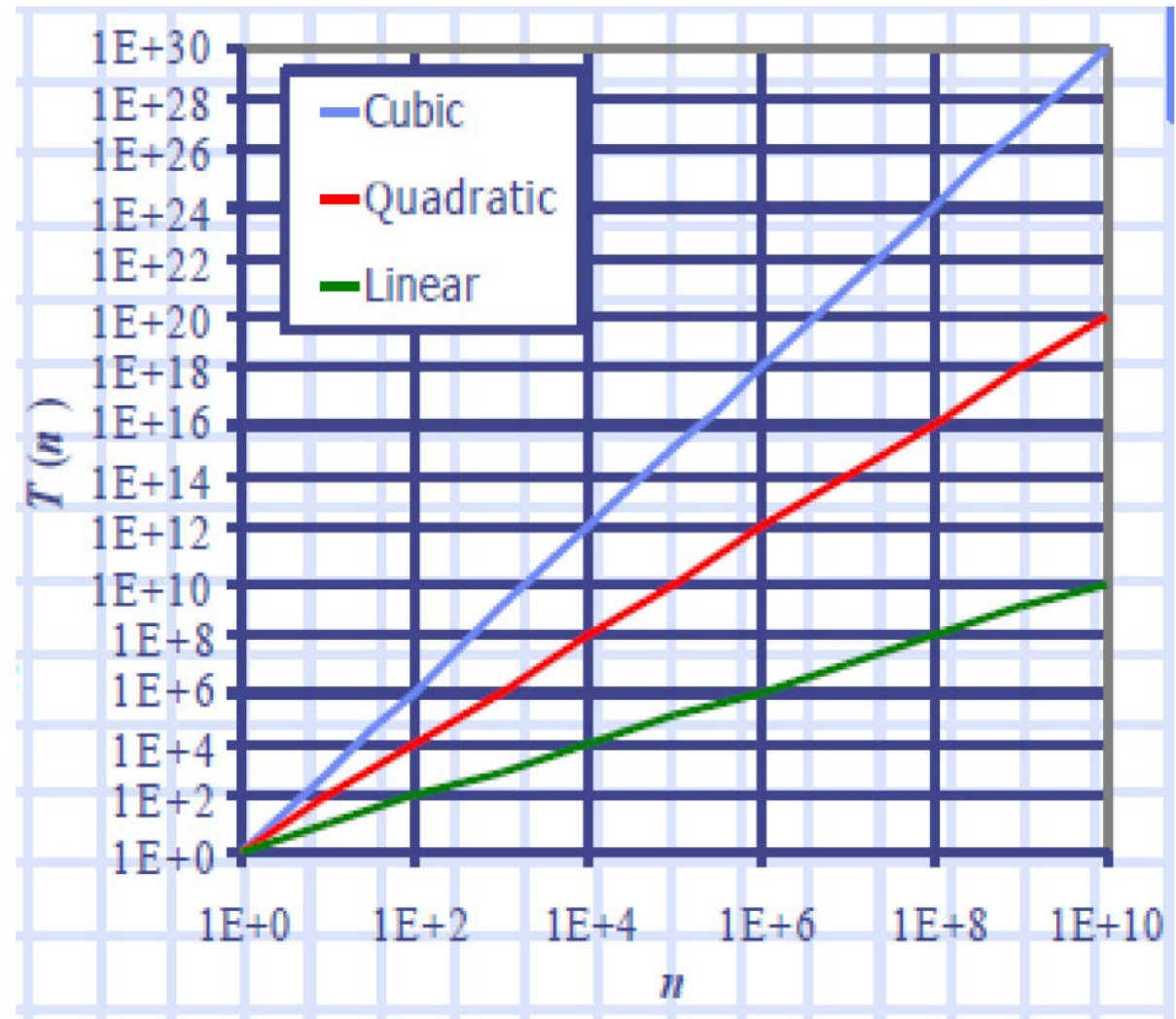
An algorithm with an asymptotically slow running time is beaten in the long run by an algorithm with an asymptotically faster running time.

Growth rate

Growth rates of functions:

- Linear $\approx n$
- Quadratic $\approx n^2$
- Cubic $\approx n^3$

In a log-log chart, the slope of the line corresponds to the growth rate of the function



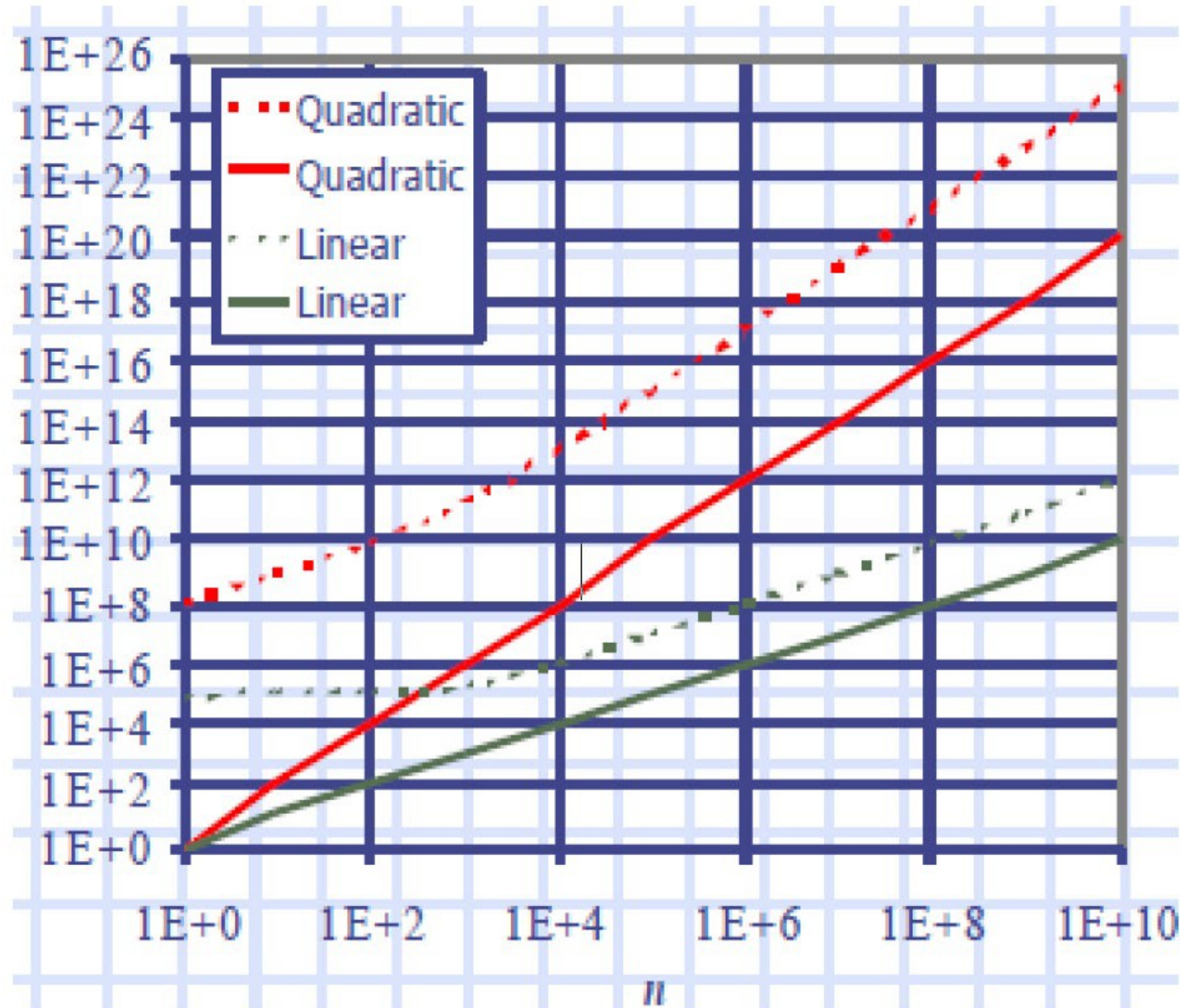
Growth rate

The growth rate is not affected by

- constant factors or
- lower-order terms

Examples

- $10^2n + 10^5$ is a linear function
- $10^5n^2 + 10^8n$ is a quadratic function



“Big-Oh” Notation

“Big-Oh” notation is probably the most commonly used form of asymptotic notation.

- ❖ Given two positive functions $f(n)$ and $g(n)$ (defined on the nonnegative integers), we say $f(n)$ is $O(g(n))$, written $f(n) \in O(g(n))$, if there are constants c and n_0 such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$

“Big-Oh” Notation

- ❖ Given two positive functions $f(n)$ and $g(n)$ (defined on the nonnegative integers), we say $f(n)$ is $O(g(n))$, written $f(n) \in O(g(n))$, if there are constants c and n_0 such that:

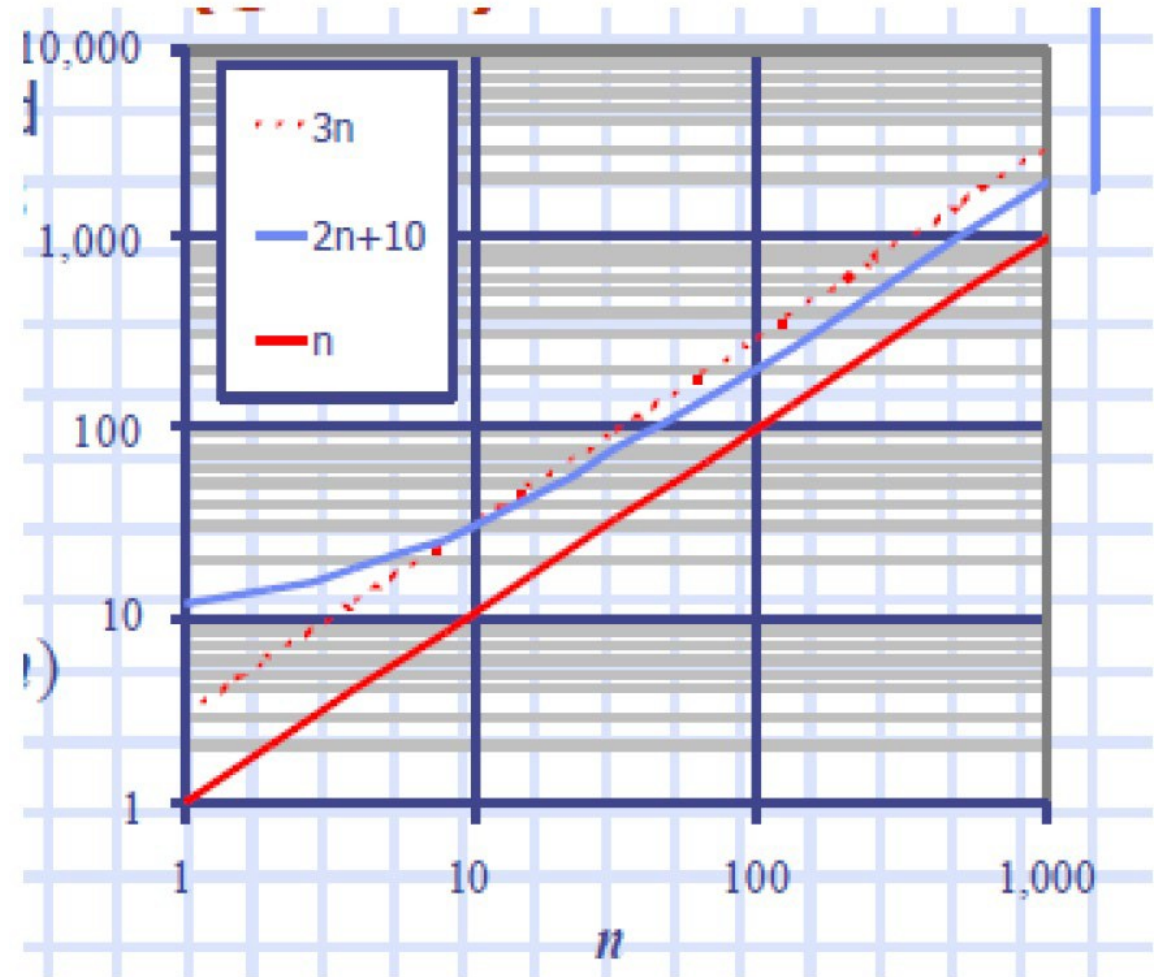
$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$

Example: $2n + 10$ is $O(n)$

“Big-Oh” Notation

Example: $2n + 10$ is $O(n)$

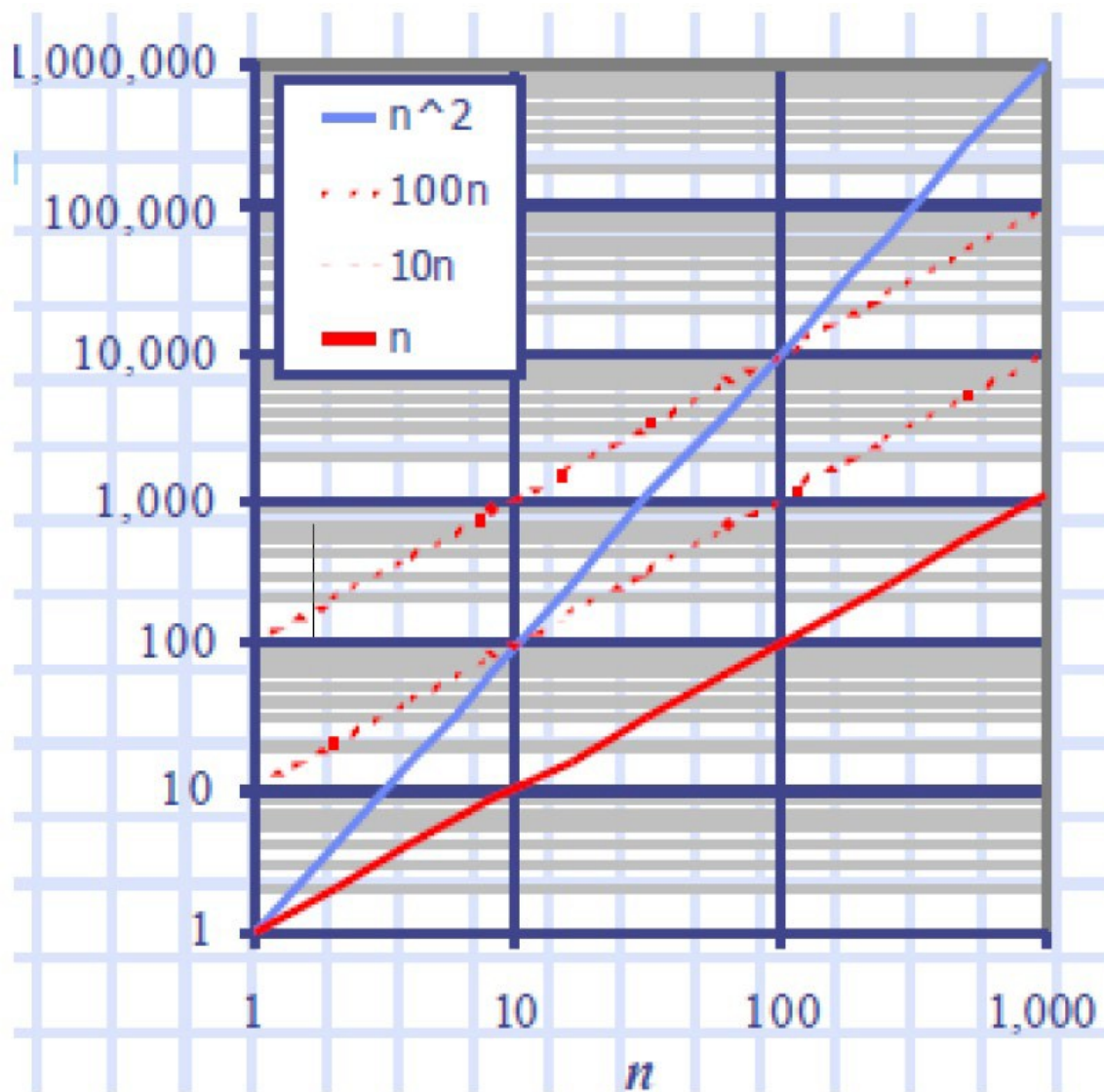
- $2n + 10 \leq cn$
- $(c - 2)n \geq 10$
- $n \geq 10/(c - 2)$
- Pick $c = 3$ and $n_0 = 10$



“Big-Oh” Notation

Example: the function n^2 is not $O(n)$

- $n^2 \leq cn$
- $n \leq c$
- The above inequality cannot be satisfied since c must be a constant



Growth rates (running time)

Functions ordered by growth rate:

$\log_2 n$	$N^{1/2}$	n	$n \log_2 n$	n^2	n^3	2^n
1	1.4	2	2	4	8	4
2	2	4	8	16	64	16
3	2.8	8	24	64	512	256
4	4	16	64	256	4096	65536
5	5.7	32	160	1024	32768	4294967296
6	8	64	384	4096	262144	1.84^{19}
7	11	128	896	16384	2097152	2.40×10^{38}
8	16	256	2048	65536	16777216	1.15^{77}
9	23	512	4608	262144	134217728	1.34×10^{154}
10	32	1024	10240	1048576	1073741824	1.79^{308}

More “Big-Oh” Examples

- $7n-2$

$7n-2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq cn$ for $n \geq n_0$

this is true for $c = 7$ and $n_0 = 1$

- $3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq cn^3$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 21$

Big-Oh and Growth Rate

- The big-Oh notation gives an upper bound on the growth rate of a function.
- The statement “ $f(n)$ is $O(g(n))$ ” means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$.
- We can use the big-Oh notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Common functions

Here is a list of classes of functions that are commonly encountered when analyzing algorithm

- *Constant* $O(1)$
- *Logarithmic* $O(\log n)$
- *Linear* $O(n)$
- *Log-linear* $O(n \log n)$
- *Quadratic* $O(n^2)$
- *Cubic* $O(n^3)$
- *Polynomial* $O(n^k)$
- *Exponential* $O(a^n), a > 1$
- *Factorial* $O(n!)$

Big-Oh Rules

$2n$ is $O(n^2)$?

- If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,
 - Drop lower-order terms
 - Drop constant factors
- Use the smallest possible class of functions
 - Say “ $2n$ is $O(n)$ ” instead of “ $2n$ is $O(n^2)$ ”
- Use the simplest expression of the class
 - Say “ $3n + 5$ is $O(n)$ ” instead of “ $3n + 5$ is $O(3n)$ ”

Further examples of Big-Oh

1. $13n^3 + 7n \log n + 3$ is $O(n^3)$.

Proof: $13n^3 + 7n \log n + 3 \leq 16n^3$, for $n \geq 1$

2. $3 \log n + \log \log n$ is $O(\log n)$.

Proof: $3 \log n + \log \log n \leq 4 \log n$, for $n \geq 2$

3. 2^{70} is $O(1)$.

Proof: $2^{70} \leq 2^{70} * 1$, for $n \geq 1$.

Asymptotic Algorithm Analysis

The asymptotic analysis of an algorithm determines the running time in big-Oh notation

To perform the asymptotic analysis

- We find the worst-case number of primitive operations executed as a function of the input size
- We express this function with big-Oh notation

Example:

- We determine that the algorithm “Maximum-Element(A) ” executes at most $7n - 2$ primitive operations
- We say that algorithm “runs in $O(n)$ time”

Asymptotic Algorithm Analysis: Example

Algorithm *prefixAverages1*(X, n)

Input array X of n integers

Output array A of prefix averages of X #operations

$A \leftarrow$ new array of n integers n

for $i \leftarrow 0$ **to** $n - 1$ **do** n

$s \leftarrow X[0]$ n

for $j \leftarrow 1$ **to** i **do** $1 + 2 + \dots + (n - 1)$

$s \leftarrow s + X[j]$ $1 + 2 + \dots + (n - 1)$

$A[i] \leftarrow s / (i + 1)$ n

return A 1

Algorithm prefixAverage1 runs in $O(?)$

Asymptotic Algorithm Analysis: Example

- ◆ The running time of *prefixAverages1* is $O(1 + 2 + \dots + n)$
- ◆ The sum of the first n integers is $n(n + 1) / 2$
- ◆ Thus, algorithm *prefixAverages1* runs in $O(n^2)$ time
- ◆ Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Asymptotic Algorithm Analysis: Example

Algorithm <i>prefixAverages2</i> (X, n)	
Input array X of n integers	
Output array A of prefix averages of X	#operations
$A \leftarrow$ new array of n integers	n
$s \leftarrow 0$	1
for $i \leftarrow 0$ to $n - 1$ do	n
$s \leftarrow s + X[i]$	n
$A[i] \leftarrow s / (i + 1)$	n
return A	1

Algorithm prefixAverage2 $O(n)$

Asymptotic Algorithm Analysis: Exercises

- 1 Give a **big-Oh** characterization, in terms of n , of the running time of the method Loop1.
- 2 Perform a similar analysis for method Loop2.
- 3 Perform a similar analysis for method Loop3.
- 4 Perform a similar analysis for method Loop4.

Algorithm Loop1(n):

```
1   $s \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $n$  do
3       $s \leftarrow s + i$ 
```

Algorithm Loop2(n):

```
1   $p \leftarrow 1$ 
2  for  $i \leftarrow 1$  to  $2n$  do
3       $p \leftarrow p \cdot i$ 
```

Algorithm Loop3(n):

```
1   $p \leftarrow 1$ 
2  for  $i \leftarrow 1$  to  $n^2$  do
3       $p \leftarrow p \cdot i$ 
```

Algorithm Loop4(n):

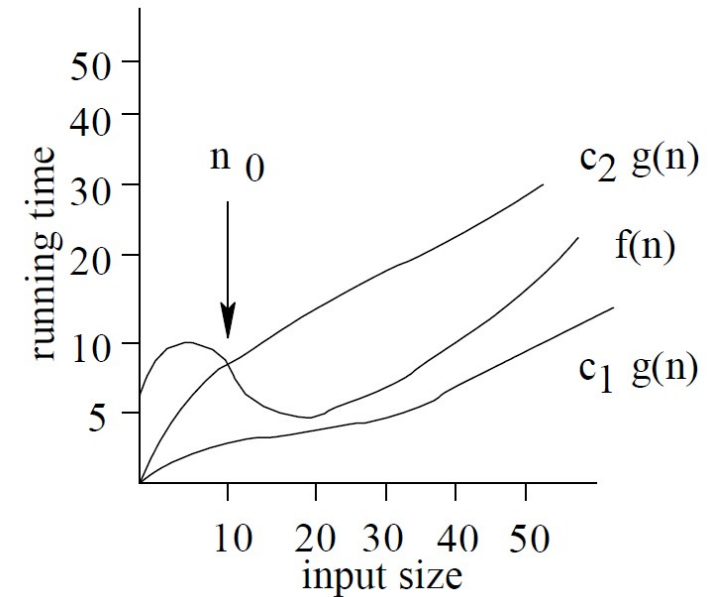
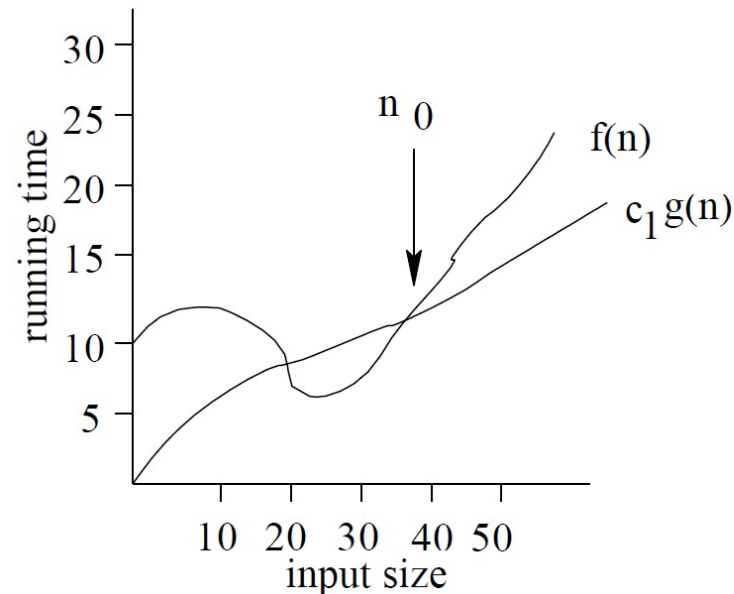
```
1   $s \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $2n$  do
3      for  $j \leftarrow 1$  to  $i$  do
4           $s \leftarrow s + i$ 
```

$\Omega(n)$ and $\Theta(n)$ notation

- ❖ We say that $f(n)$ is $\Omega(g(n))$ (*big-Omega*) if there are real constants c and n_0 such that:

$$f(n) \geq cg(n) \text{ for all } n \geq n_0.$$

- ❖ We say that $f(n)$ is $\Theta(g(n))$ (*Theta*) if $f(n)$ is $\Omega(g(n))$ and $f(n)$ is also $O(g(n))$.



Intuition for Asymptotic Notation

- Big-Oh

$f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically less than or equal to $g(n)$

- Big-Omega

$f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically greater than or equal to $g(n)$

- Big-Theta

$f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically equal to $g(n)$

Examples

1. $3\log n + \log \log n$ is $\Omega(_\log n__)$.

Proof: $3\log n + \log \log n \geq 3 \log n$, for $n \geq 2$.

2. $3\log n + \log \log n$ is $\Theta(_\log n__)$.

Space Complexity

- Space complexity is a measure of the amount of working storage an algorithm needs. That means how much memory, in the worst case, is needed at any point in the algorithm.
- As with time complexity, we're mostly concerned with how the space needs grow, in big-Oh terms, as the size N of the input problem grows.

Space Complexity

```
int sum(int x, int y, int z) {  
    int r = x + y + z;  
    return r;  
}
```

requires 3 units of space for the parameters and 1 for the local variable, and this never changes, so this is $O(1)$.

Space Complexity

```
int sum(int a[], int n) {  
    int r = 0;  
    for (int i = 0; i < n; ++i) {  
        r += a[i];  
    }  
    return r;  
}
```

requires N units for a, plus space for n, r and i, so it's $O(N)$.